Neima Bravo

GitHub: nbravo.dev@outlook.com | UTEP: nrbravo@miners.utep.edu

Submitted on October 13, 2026.

CS 4475 – Operating Systems Concepts - Fall 2026

Dr. Shirley Moore

- Lab Report –

PA 2: Priority-based Scheduler for xv6

Repository: https://github.com/datanbravo/xv6-riscv

Branch: pa2

Task 1. Implement the getprocs() System Call

For this task, I implemented the getprocs() system call to retrieve information about the processes currently in the system. The system call goes through the process table, collects information about each active process, and copies it into an array that can be accessed by a user program.

I added the pstat structure in kernel/pstat.h to store the process ID, state, memory size, parent process ID, and name. I also added the necessary system call declarations and connections so that the ps command could access this information.

The main files modified were kernel/pstat.h, kernel/proc.c, kernel/defs.h, kernel/sysproc.c, kernel/syscall.h, kernel/syscall.c, user/user.h, user/usys.pl, and user/ps.c. I also updated the Makefile to include the ps program.

To test the implementation, I ran the ps command in xv6. The output displayed three processes: init, sh, and ps. Each process showed its ID, state, memory size, parent process ID, and name.

The results confirmed that the system call was retrieving process information and displaying it through the ps command.

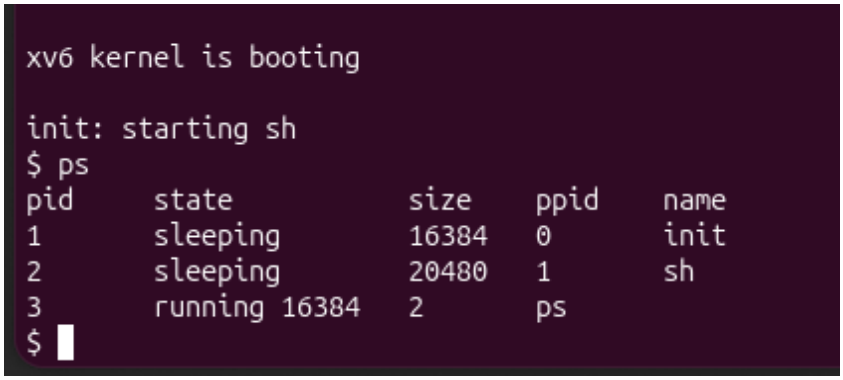


Figure 1. Output of the ps command displaying active processes in xv6.

Task 2. Implement the Priority System Calls

For this task, I added a priority field to struct proc and implemented the getpriority() and setpriority() system calls. The priority is initialized to 0 when a process is created, and child processes inherit their parent's priority.

The getpriority() system call returns the current process's priority, while setpriority() allows it to change its priority to a value between 0 and 49. If an invalid value is provided, the system call returns -1 without changing the existing priority.

I also modified the getprocs() system call to include each process's priority in the pstat structure. Then, I updated user/ps.c to display the new priority column alongside the existing process information.

The main files modified were kernel/proc.h, kernel/proc.c, kernel/pstat.h, kernel/defs.h, kernel/sysproc.c, kernel/syscall.h, kernel/syscall.c, user/user.h, user/usys.pl, and user/ps.c.

To test the implementation, I ran the modified ps command and created a prioritytest program. The initial output showed that init, sh, and ps all had a priority of 0.

The prioritytest program confirmed that a process could retrieve its original priority, change it to 25, and reject an invalid priority of 50. I also tested priority inheritance using fork(), which showed that both the parent and child had a priority of 25.

Finally, I tested the boundary values. Priorities 0 and 49 were accepted, while -1 was rejected. The process retained its previous priority after the invalid request.

```
xv6 kernel is booting

init: starting sh
$ ps
pid     state           size    ppid    priority        name
1       sleeping        16384   0       0       init
2       sleeping        20480   1       0       sh
3       running 16384   2       0       ps
$
```

Figure 2.1 Modified ps command

```
xv6 kernel is booting

init: starting sh
$ prioritytest
Original priority: 0
Updated priority: 25
Invalid priority 50 rejected correctly
Final priority: 25
$
```

Figure 2.2 Priority changes and validation

```
xv6 kernel is booting

init: starting sh
$ prioritytest
Original priority: 0
Updated priority: 25
Invalid priority 50 rejected correctly
Final priority: 25
Child priority: 25
Parent priority: 25
$
```

Figure 2.3 Priority inheritance

```
xv6 kernel is booting

init: starting sh
$ prioritytest
Original priority: 0
Updated priority: 25
Invalid priority 50 rejected correctly
Final priority: 25
Child priority: 25
Parent priority: 25
Set priority 0: 0
Priority after setting 0: 0
Set priority 49: 0
Priority after setting 49: 49
Set priority -1: -1
Priority after invalid value: 49
$
```

Figure 2.4 Boundary value testing

Task 3. Track Process Ready Time and Display Age

For this task, I added a readytime field to struct proc to keep track of when a process becomes ready to run. This field is initialized to 0 when a process is allocated and updated to the current system tick whenever its state changes to RUNNABLE.

I updated kernel/proc.c to assign the current value of ticks to readytime whenever a process enters the RUNNABLE state, including when a process is created, yields the CPU, or wakes up from sleeping.

I also modified kernel/pstat.h and the getprocs() implementation to make the ready time available to the ps command. The process age is calculated by subtracting its readytime from the current system tick count.

Process age = current ticks - readytime

The ps command was modified to display this age only for processes in the RUNNABLE state. For processes that are sleeping or running, a dash is displayed instead.

The main files modified were kernel/proc.h, kernel/proc.c, kernel/pstat.h, and user/ps.c.

To test the implementation, I ran multiple matmul processes in the background and used ps to check their states and ages. The output showed several processes in the RUNNABLE state with different ages, ranging from 0 to 8 ticks. The sleeping and running processes displayed a dash instead of an age.

These results confirmed that the system was tracking when processes became ready and calculating their waiting time correctly at the moment ps collected the information.

```
$ matmul 100 &
matmul 100 &
ps
$ $ pid state          size    age     ppid    priority        name
1       sleeping       16384   -       0       0               init
2       sleeping       20480   -       1       0               sh
28      running 16384  -       2       0               ps
5       runnable       16384   8       1       0               matmul
7       runnable       16384   7       1       0               matmul
10      runnable       16384   6       1       0               matmul
12      runnable       16384   5       1       0               matmul
15      runnable       16384   4       1       0               matmul
17      runnable       16384   3       1       0               matmul
20      runnable       16384   2       1       0               matmul
25      runnable       16384   1       1       0               matmul
27      runnable       16384   0       1       0               matmul
$
```

Figure 3.1 Process age displayed by the modified ps command

Task 4. Implement a Priority-Based Scheduler

For this task, I modified the xv6 scheduler to support priority-based scheduling while keeping the original round-robin scheduler available. I added constants in kernel/param.h to select the scheduling policy at compile time.

The main changes were made in kernel/proc.c, specifically in the scheduler() function. When priority scheduling is enabled, the scheduler searches the process table for RUNNABLE processes and selects the one with the highest priority. If multiple processes have the same priority, any of them can be selected. The original round-robin scheduler remains available when that scheduling policy is selected.

I also used the Makefile to configure xv6 to run with one CPU, as required by the assignment.

The main files modified were kernel/param.h, kernel/proc.c, and the Makefile. I also created user/schedtest.c to test the scheduler.

To verify the implementation, I created three processes with priorities 10, 20, and 30. In one test, the processes executed in the expected priority order, with priority 30 starting and finishing first, followed by priority 20 and finally priority 10.

In another test, the processes showed interleaved execution instead of following a strictly sequential priority order. This demonstrated how process timing and the availability of RUNNABLE processes can affect the observed results.

One difficulty I encountered was understanding why the execution order was not always consistent between tests. I learned that testing a scheduler requires considering when processes become ready, rather than assuming that every process is available for selection at the same time.

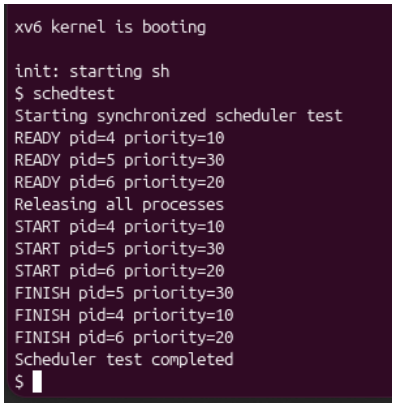


Figure 4.1 Correct priority order

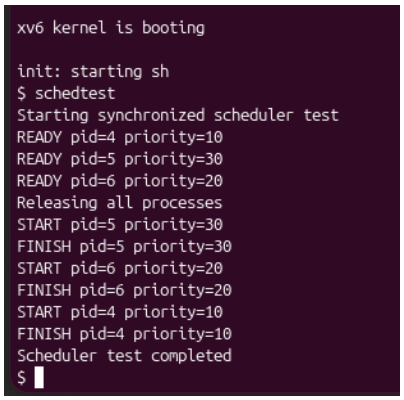


Figure 4.2 Interleaved execution

Task 5. Implement Priority Aging

For this task, I implemented priority aging to reduce the possibility of starvation in the priority-based scheduler. The aging policy increases a process's effective priority by one for

every five ticks it spends waiting in the RUNNABLE state. The effective priority is limited to 99, while the process's original priority remains unchanged.

I used the readytime field from Task 3 to calculate how long each process has been waiting. I also added the AGING_ENABLED configuration in kernel/param.h so aging can be enabled or disabled when compiling the scheduler.

The main files modified were kernel/param.h and kernel/proc.c. I also created user/agingtest.c to compare scheduling behavior with aging enabled and disabled.

For testing, I created three processes with priorities 10, 10, and 9. I used the same workload for both configurations to compare their execution order.

With aging disabled, the processes executed sequentially. The first priority-10 process started and finished, followed by the second priority-10 process. The priority-9 process did not start until both higher-priority processes had finished.

With aging enabled, all three processes started before any of them finished. The priority-9 process was able to begin executing while the two priority-10 processes were still active. This showed that enabling aging changed the scheduling behavior and allowed the lower-priority process to receive CPU time earlier in the test.

One difficulty I encountered was finding a test that demonstrated the effects of aging. My initial tests used priorities that were too far apart, so the higher-priority processes finished before the lower-priority process accumulated enough waiting time. I adjusted the priorities and increased the workload to make the comparison more meaningful.

Through this task, I learned how aging can improve fairness in priority scheduling and how important it is to design tests that actually demonstrate the behavior being evaluated. Although the results support that aging influenced scheduling, they do not prove that starvation is prevented in every situation.

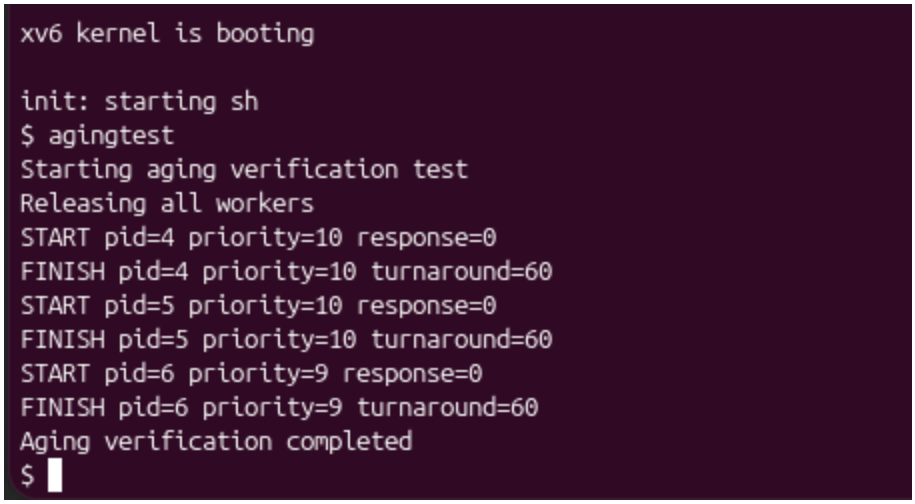


Figure 5.1 Aging disabled

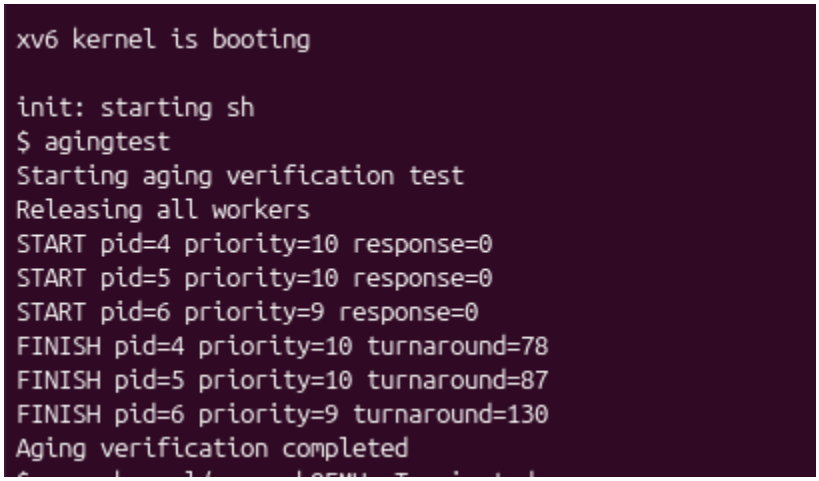


Figure 5.2 Aging enabled

Performance evaluation

Measurement	Aging disabled	Aging enabled
Average turnaround	17.67 ticks	17.00 ticks
Average response	8.67 ticks	8.67 ticks

I also compared average turnaround and response times using schedtest. With aging disabled, the average turnaround time was 17.67 ticks and the average response time was 8.67 ticks. With aging enabled, the average turnaround time was 17 ticks, while the average response time remained at 8.67 ticks. The difference was small, so these measurements alone were not enough to confirm a consistent performance improvement.

Task 6. Summary

Through this assignment, I learned more about how xv6 manages processes and how different scheduling policies affect their execution. I learned how to create system calls to retrieve and modify process information, track how long processes wait in the RUNNABLE state, and modify the scheduler to select processes based on their priority.

One of the main difficulties I encountered was testing the priority scheduler and aging policy. At first, some of the results did not show the expected execution order, which made it difficult to determine whether the implementation was working correctly. I had to adjust the test programs, synchronize the processes, and compare different configurations to better understand the results.

The aging policy was particularly challenging because my initial tests did not demonstrate its effects. I eventually adjusted the process priorities and increased the workload, which allowed me to observe the difference between having aging enabled and disabled. I also corrected the effective-priority limit to match the assignment requirements.

Overall, this assignment helped me understand how scheduling decisions are made and how factors such as priority and waiting time affect process execution. It also showed me the importance of testing different scenarios instead of assuming that a successful execution means the implementation is completely correct.

AI Assistance Disclosure

I used ChatGPT as a support tool throughout this assignment. It helped me understand the requirements, identify the files and functions involved in implementing system calls and scheduling policies, and develop and troubleshoot parts of the implementation.

I also used ChatGPT to help investigate compilation errors, compare scheduling results, and identify issues with the aging implementation. This included correcting the effective-priority limit and adjusting the aging tests to better demonstrate scheduling differences.